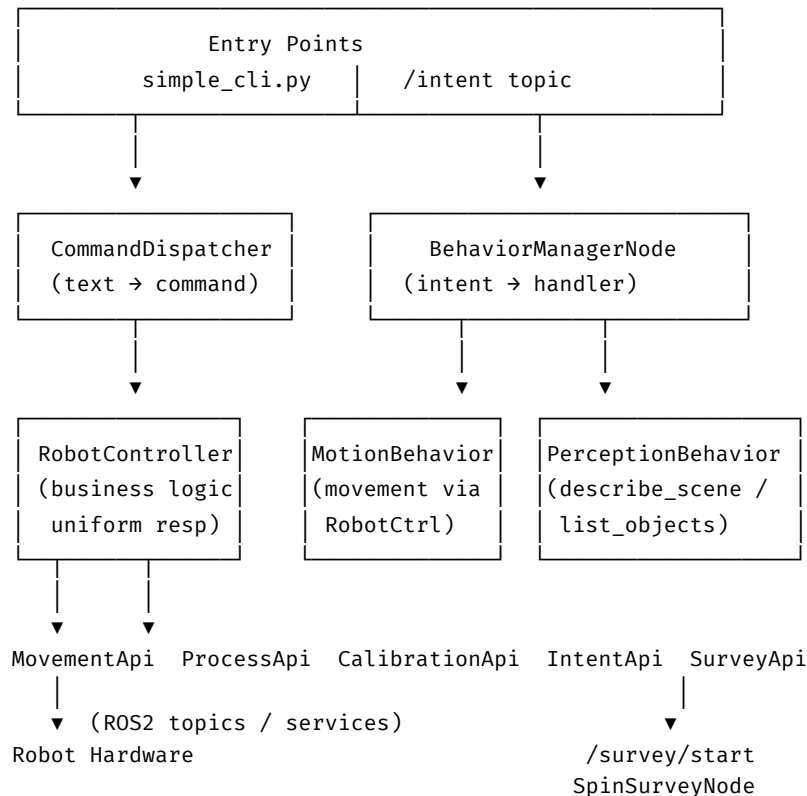


dome_control Package: Theory of Operation

What This Is

`dome_control` is a ROS2 package that gives a mobile robot a voice-driven command interface. A user speaks a command; a sibling package (`dome_voice`) transcribes and maps the intent, then publishes it to `/intent`. This package receives that intent, routes it to a domain behavior handler, and executes motion or perception actions — all over ROS2 topics and services. There is also a CLI for scripted or remote control.

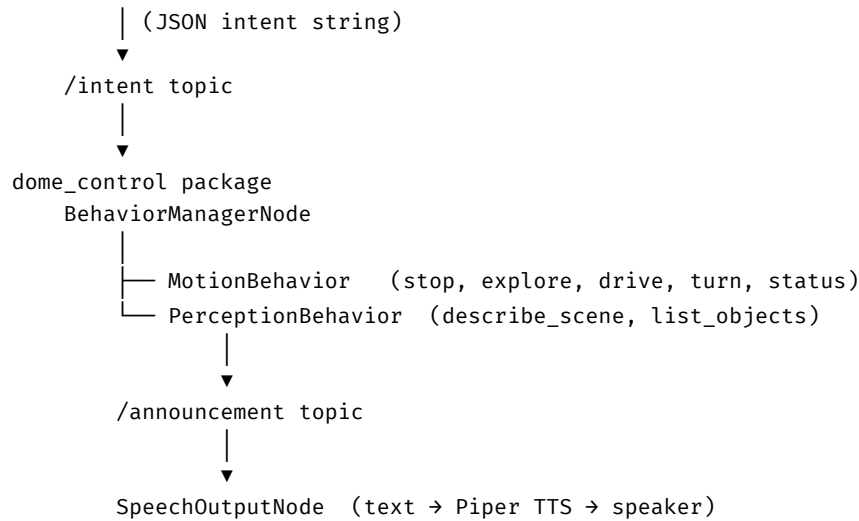
Architecture in Layers



Intent Flow

Voice input lives in the sibling package `dome_voice`. Its pipeline produces JSON intent strings on `/intent`. This package consumes them:

`dome_voice` package
 `VoiceInputNode`



Two Execution Paths

CommandDispatcher routes to one of two paths:

Behavior path: Commands that involve intent semantics (scene describe, scene objects, intent verbs) publish JSON to /intent via IntentPublisher. Query intents wait up to 5s for a reply on /announcement.

Direct path: All other commands (move, launch, status, calibrate) call RobotController methods synchronously and return a CommandResponse.

Configuration Foundation

Everything starts with ConfigManager. It loads a YAML file, coerces types, manages subdirectory creation, and detects test vs. production environments. Nearly every other module takes a ConfigManager instance at construction. Read 01-config_manager first.

Vision Integration

BehaviorManagerNode optionally loads class profiles from dome_vision at startup:

```
self.declare_parameter("class_profiles_path", DEFAULT_VISION_CONFIG)
# loads self.profiles and self.label_map
```

PerceptionBehavior uses these to map raw vision model class IDs to human-readable display names. If dome_vision is absent, the node starts normally and falls back to raw class IDs.

Command Object Hierarchy

Commands are dataclasses organized in a hierarchy:

```
BaseCommand
├── MovementCommand    (drive, spin, stop)
├── LaunchCommand      (start/stop ROS launch files)
├── SystemCommand      (status, shutdown)
├── NavigationCommand  (map operations)
└── IntentCommand      (behavior path commands)
```

CommandDispatcher parses text into these objects. RobotController executes them. New entry points share the same execution path.

Launch Structure

Each package owns its own launch files:

Package	Launch	What it starts
dome_control	robot.launch.py	speech_output, behavior_manager
dome_control	remote.launch.py	optional behavior_manager
dome_voice	robot.launch.py	voice_input
dome_vision_ros	robot.launch.py	oak camera, semantic_map

Reading Order

#	File	What it covers
01	config_manager	YAML config, factory pattern, type coercion
02	base_api	ROS2 node base class, shared plumbing
03–06	movement/intent/calibration/robot_controller	Four ROS2 API nodes
07	announcement_controller	Shared message contract for /announcement
08–10	movement/launch/system_command_classes	System command classes by domain
11	robot_controller	Business logic, lazy API init, uniform responses
12	command_dispatcher	Text → command object parsing
13	speech_api	CLI-side announcement publisher
14	simple_cli	CLI entry point, REPL and non-interactive modes
18	behavior_manager (IntentParser)	Pure JSON→Intent parsing
19	behavior_manager_ros	ROS2 intent router, vision profile loading
20	voice_input_node	(orphaned — source moved to dome_voice)
21	speech_output_node	Piper TTS, /announcement consumer
22	motion_behavior	Motion intent execution via RobotController

#	File	What it covers
23	perception_behavior	describe_scene service call, list_objects from cache
24	intent_publisher	JSON intent publishing with query reply wait
25	spin_survey	360° spin state machine, no ROS2
26	spin_survey_node	SpinSurveyNode: service-triggered tick driver
X11	survey_api	SurveyApi: /survey/start client
X12	survey_commands	survey.start CLI command definition

Appendices (X01–X12) cover parameter definitions, command schemas, stub modules, and survey API.

Key Design Decisions

Uniform return type. Every `RobotController` method returns `CommandResponse(success, message, data)`. No exceptions escape to callers.

Lazy node initialization. ROS2 nodes are expensive. APIs are constructed on first use, not at startup.

Intent over raw commands. Voice input goes through `IntentParser` → `IntentCommand` → `BehaviorManagerNode` rather than directly to movement commands. Decouples speech vocabulary from robot action set.

Unconditional service call. `PerceptionBehavior.call_describe_scene` calls `/describe_scene` even when `service_is_ready()` is `False`, to handle transient startup delays. Future silently stalls if service never appears.

Vision degradation. Both `dome_vision` import and label map loading are wrapped in `try/except`. System runs without vision; perception responses degrade to raw class IDs, not crashes.

ConfigManager: Persistent Configuration for the Robot

Every robot needs a place to live its settings — speed limits, directory paths, launch templates, and runtime flags. `ConfigManager` is that place. It is the first dependency of almost every other module in the stack, which is why it sits at the top of the reading order.

The Factory Pattern for Safe Initialization

Python constructors cannot return errors, so a direct `__init__` that opens a file is hard to test and hard to reason about. `ConfigManager` solves this with a class-method factory:

```

@classmethod
def create(cls, config_file: str) → "ConfigManager":
    manager = cls(config_file)
    manager.load_config()
    manager.ensure_subdirs()
    manager.detect_test_environment()
    return manager

```

create sequences the steps that must happen before the object is usable: load the YAML, create missing directories, and detect if we are running under pytest. Callers always use `create`; the constructor is intentionally bare — just stores paths and initializes variables to an empty dict.

Variables as a Typed Dictionary

All configuration lives in a single flat dict `self.variables`, loaded from YAML. The smart part is `set_variable`, which coerces string input to the right Python type before storing:

```

def set_variable(self, name: str, value: object):
    if not isinstance(value, str):
        self.variables[name] = value
    elif value.lower() in ["true", "false"]:
        self.variables[name] = value.lower() == "true"
    elif self.is_number(value):
        self.variables[name] = self.convert_number(value)
    else:
        self.variables[name] = value
    self.save_config()

```

The priority order is: non-string passes through, then bool detection, then numeric detection, then raw string. This matters because YAML already types most values — the coercion path is only exercised when values arrive as strings from the CLI.

```

def convert_number(self, value: str) → int | float:
    if "." in value:
        return float(value)
    return int(value)

```

The dot-presence heuristic for int vs float is simple and correct for robot config values.

Path Resolution

The robot stores maps, logs, and other artifacts relative to the config directory. `resolve_path` handles the ambiguity between absolute and relative paths:

```
def resolve_path(self, path_str: str) → Path:
    path = Path(path_str).expanduser()
    if path.is_absolute():
        return path
    return (self.control_dir / path).resolve()
```

ensure_subdirs calls this to create the maps/ and logs/ directories on first use, so callers never have to check.

Test Environment Detection

A subtle but important design: `detect_test_environment` sets `dry_run = True` when `pytest` or `unittest` is present in `sys.modules`. This propagates to `MovementApi.cmd_vel_helper`, which skips real hardware commands in tests without requiring explicit mocking of the hardware layer.

```
def detect_test_environment(self):
    if "pytest" in sys.modules or "unittest" in sys.modules:
        self.variables["dry_run"] = True
```

Launch Templates

The config file also carries launch templates — named command templates with default parameters. `ConfigManager` exposes these as a plain dict; `ProcessApi` consumes them to build `LaunchConfig` objects. The separation keeps `ConfigManager` free of any subprocess knowledge.

```
def get_launch_templates(self) → dict:
    return self.variables.get("launch_templates", {})
```

Observations

- **No schema validation.** Missing required keys (e.g., `linear_speed`) surface as `None` from `get_variable`, caught only at point of use. A startup `validate()` would catch config errors earlier.
- **Save on every set.** `set_variable` calls `save_config()` unconditionally. A transactional API or explicit `flush()` would be more efficient for bulk updates.
- **Flat namespace.** All variables are top-level. Nested sections (hardware, nav, speech) would make the config file more readable but require deeper access helpers.

BaseApi

`base_api.py` defines the abstract base class shared by all ROS2 API nodes in this package. It is the point where ROS2's `Node` machinery meets the package's configuration and logging conventions.

Multiple Inheritance

```
class BaseApi(Node, ABC):
    def __init__(self, node_name: str, config_manager: cm.ConfigManager = None):
        super().__init__(node_name)
        self.config = config_manager
```

`BaseApi` inherits from both `Node` (ROS2) and `ABC` (Python abstract base class). The `ABC` inheritance isn't strictly necessary here — there are no `@abstractmethod` declarations — but it signals that `BaseApi` is not meant to be instantiated directly.

`config_manager` may be `None`. Callers that need `config` pass it explicitly. In production all API nodes receive the same `ConfigManager` instance from `Robot-Controller`, which is important for shared state (e.g., `dry_run` flag, speed variables).

Logging Conventions

Four logging methods wrap ROS2's logger with a twist:

```
def log_debug(self, message: str):
    self.get_logger().debug(message)

def log_info(self, message: str):
    print(message)           # ← goes to stdout, not ROS2 log

def log_warn(self, message: str):
    self.get_logger().warn(message)

def log_error(self, message: str):
    self.get_logger().error(message)
```

`log_info` routes to `print()` rather than the ROS2 logger. This is intentional: the CLI user sees `stdout` directly in their terminal; ROS2 info-level log output typically goes to a separate log sink and isn't visible interactively. `debug`, `warn`, and `error` use the ROS2 logger so they appear in `ros2 topic echo /rosout` and log files.

Bounds Checking

```
def check_bounds(self, value: float, min_val: float, max_val: float, name: str) → bool:
    if not (min_val ≤ value ≤ max_val):
        self.log_warn(f"{name} out of bounds: {value}. Must be [{min_val}, {max_val}]")
        return False
    return True
```

This helper is used by `MovementApi.check_velocity_limits` before every velocity command. Keeping it in `BaseApi` makes it available to any future API class

that also needs range validation.

Node Lifecycle

`destroy_node` overrides the ROS2 default to log a shutdown message before calling `super().destroy_node()`. This gives a consistent teardown trace in logs when the CLI exits.

Observations

- `config_manager` is stored as-is (may be `None`). Subclasses that access `self.config` without checking for `None` will crash if called without a config. Callers that don't need config (e.g. `IntentApi`) pass `None` safely as long as they don't call config-dependent methods.
- `ABC` is imported but not used for its primary purpose. Future API classes that must implement specific methods could use `@abstractmethod` here.

MovementApi: Velocity Control and Odometry

`MovementApi` is the thin ROS2 layer between the robot's wheels and the command layer. It publishes `Twist` messages to `/cmd_vel` and subscribes to `/odom`. All movement commands in the stack eventually call one of its three primitives: `cmd_vel_helper`, `turn_amount`, or `stop`.

Inheriting from BaseApi

`MovementApi` extends `BaseApi`, which wraps `rclpy.Node`. This gives it access to `create_publisher`, `create_subscription`, `log_info`, and the `config` property without re-implementing node boilerplate.

```
class MovementApi(base.BaseApi):
    def __init__(self, config_manager: cm.ConfigManager = None):
        super().__init__("movement_api", config_manager)
        self.cmd_vel_pub = self.create_publisher(Twist, "/cmd_vel", 1)
        self.odom_sub = self.create_subscription(
            Odometry, "/odom", self.odom_callback, 10
        )
        self._validate_config()
        self.current_pose = None
```

A queue depth of 1 for `cmd_vel` is intentional: if the robot can't keep up with commands, the newest command should win, not queue behind stale ones.

Speed as Config Properties

Rather than capturing speeds at construction time, each speed is a live property reading from `ConfigManager`. This means `config set linear_speed 0.4` takes

effect on the next movement call without restarting the node:

```
@property
def linear(self) → float:
    return self.config.get_variable("linear_speed")
```

The same pattern applies to angular, linear_min, linear_max, angular_min, angular_max.

Config Validation at Startup

The six speed variables are required. `_validate_config` checks all of them at construction time so failures are loud and early rather than silent `None` math errors mid-movement:

```
def _validate_config(self):
    required_vars = ["linear_speed", "angular_speed",
                    "linear_min", "linear_max", "angular_min", "angular_max"]
    missing = [v for v in required_vars if self.config.get_variable(v) is None]
    if missing:
        raise ValueError(f"Missing required configuration variables: {' '.join(missing)}")
```

The Central Primitive: `cmd_vel_helper`

All movement boils down to this method — publish a Twist at 10 Hz for a given duration, then stop:

```
def cmd_vel_helper(self, linear: float, angular: float, seconds: float):
    if not self.blocking_ok:
        raise RuntimeError("cmd_vel_helper called from a non-blocking context")
    if not self.check_velocity_limits(linear, angular):
        return
    if self.config.is_dry_run():
        self.log_info(f"DRY RUN: Would move linear={linear}, angular={angular} for {seconds}s")
        return

    twist = Twist()
    twist.linear.x = linear
    twist.angular.z = angular
    start_time = time.time()
    rate_hz = 10
    sleep_duration = 1.0 / rate_hz
    while rclpy.ok() and (time.time() - start_time) < seconds:
        self.cmd_vel_pub.publish(twist)
        time.sleep(sleep_duration)

    twist.linear.x = 0.0
```

```
twist.angular.z = 0.0
self.cmd_vel_pub.publish(twist)
```

The 10 Hz re-publish loop is needed because many ROS2 motor drivers treat a stale /cmd_vel as a stop command. The explicit zero-velocity stop at the end is a safety measure.

Why Not `spin_once`?

An earlier version called `rclpy.spin_once(self, timeout_sec=sleep_duration)` inside the loop. This crashed with `RuntimeError: Executor is already spinning` when `cmd_vel_helper` was invoked from within a `BehaviorManagerNode` callback (already being spun by `rclpy.spin`). Publishers do not need executor spinning — only subscribers do. Removing `spin_once` and replacing it with `time.sleep` fixes the nested-spin crash. The tradeoff is that the odometry subscription (`odom_callback`) does not fire during movement.

Movement Primitives

The higher-level methods are thin wrappers:

```
def move_dist(self, distance: float):
    seconds = abs(distance) / self.linear
    actual_speed = self.linear if distance >= 0 else -self.linear
    self.cmd_vel_helper(actual_speed, 0.0, seconds)

def turn_amount(self, angle: float):
    seconds = abs(angle) / self.angular
    angular_speed = self.angular if angle >= 0 else -self.angular
    self.cmd_vel_helper(0.0, angular_speed, seconds)

def turn_degrees(self, degrees: float):
    self.turn_amount(math.radians(degrees))
```

Sign conventions: positive distance = forward, positive angle = counterclockwise (ROS standard).

Odometry Subscription

The subscription stores the latest pose but does not act on it during movement (since `odom_callback` is not serviced during `cmd_vel_helper`). This is a foundation for future closed-loop control:

```
def odom_callback(self, msg):
    self.current_pose = msg.pose.pose
```

Observations

- **Open-loop movement.** `move_dist` computes time from speed; it does not use odometry feedback. Wheel slip or hardware variation accumulates as error.
- **Blocking architecture.** `cmd_vel_helper` blocks the calling thread for the full movement duration. No other intents are processed during a turn.
- **Odometry inactive during movement.** Without `spin_once`, `odom_callback` never fires while the robot is moving. Closed-loop control would require a dedicated executor thread.
- **10 Hz fixed rate.** The publish rate is hardcoded. A configurable rate or ROS timer would be more idiomatic.

IntentApi — Publishing Intents and Awaiting Replies

Purpose

IntentApi is the CLI's bridge into the ROS2 graph. It publishes structured intent messages to `/intent` and, for query-type intents, waits synchronously for the reply that arrives on `/announcement`.

Design: Fire-and-Forget vs. Synchronous Query

Most intents (`stop`, `explore`, `turn`) are fire-and-forget — the CLI doesn't wait for a result. But query intents (`describe_scene`, `list_objects`, `get_status`, `get_help`) produce text the user expects to see printed. A clean boundary separates these two modes:

```
REPLY_INTENTS = {"describe_scene", "list_objects", "get_status", "get_help"}
REPLY_TIMEOUT_S = 5.0
```

Node Lifecycle and DDS Warmup

IntentApi extends `BaseApi` (which extends `rclpy.Node`). It is created **once at CLI startup** — not lazily per command — so DDS has time to discover the `/announcement` publisher before any command fires. Creating a fresh node per command caused the announcement to arrive before the subscription was registered.

```
class IntentApi(base.BaseApi):

    def __init__(self, config_manager: cm.ConfigManager = None):
        super().__init__("intent_api", config_manager)
        self.intent_pub = self.create_publisher(String, "/intent", 10)
        self.reply_text: str | None = None
        self.create_subscription(AnnouncementMsg, "/announcement", self.on_announcement, 10)
```

Announcement Callback

The subscription captures the `text` field from any `Announcement` message. The field is `None` between turns so the spin loop can detect a fresh reply:

```
def on_announcement(self, msg) → None:
    self.reply_text = msg.text
```

Publishing and Waiting

For non-query intents the method returns `None` immediately. For query intents it spins the node in 100ms increments until a reply arrives or the timeout expires:

```
def publish(self, name: str, source: str, slots: dict) → str | None:
    msg = String()
    msg.data = json.dumps({"name": name, "source": source, "slots": slots})
    self.intent_pub.publish(msg)
    self.log_info(f"Intent published: {msg.data}")

    if name not in REPLY_INTENTS:
        return None

    self.reply_text = None
    deadline = time.monotonic() + REPLY_TIMEOUT_S
    while time.monotonic() < deadline and self.reply_text is None:
        rclpy.spin_once(self, timeout_sec=0.1)
    return self.reply_text
```

The caller (`CommandDispatcher`) uses the returned text as the CLI response message, falling back to "Intent published: {name}" if `None`.

Flow

sequenceDiagram

```
participant CLI
participant IntentApi
participant BehaviorManager
participant Behavior
participant SpeechOutput
```

```
CLI->>IntentApi: publish("describe_scene", ...)
IntentApi->>ROS: /intent: String(JSON)
IntentApi->>IntentApi: spin_once loop (5s max)
ROS /intent->>BehaviorManager: on_intent
BehaviorManager->>Behavior: execute()
Behavior->>ROS: /announcement: Announcement(text)
ROS /announcement->>IntentApi: on_announcement → reply_text
```

```
ROS /announcement→SpeechOutput: speak text
IntentApi→CLI: return reply_text
```

Observations

- The 5-second timeout is arbitrary. A tunable constant would be cleaner.
- `reply_text` is public state shared between callback and spin loop. In a multi-threaded executor this would be a race. Safe here because `spin_once` is called from the same thread.
- If two query commands are issued before the first reply arrives, `reply_text` from the first could be consumed by the second's wait loop. Not a practical concern in a REPL but worth noting.

CalibrationApi

`calibration_api.py` provides scripted movement patterns for calibration and stress testing. It sits above `MovementApi` in the dependency chain and delegates all actual motion to it.

Structure

```
class CalibrationApi(base.BaseApi):
    def __init__(self, movement_api: movement.MovementApi, config_manager: cm.ConfigManager):
        super().__init__("calibration_api", config_manager)
        self.movement = movement_api
```

`CalibrationApi` receives an existing `MovementApi` instance rather than constructing its own. This matters because `MovementApi` is a ROS2 node (it registers publishers and subscribers). Creating a second `MovementApi` would create a second `movement_api` node with duplicate pub/sub connections.

Square Pattern

```
def run_square_pattern(self, side_length: float):
    for i in range(4):
        self.movement.move_dist(side_length)
        self.movement.turn_amount(math.pi / 2)
```

Drives four equal sides with 90-degree turns between them. Because `move_dist` and `turn_amount` are open-loop (time-based), accumulated heading error means the robot does not return to exactly the starting point. The square pattern is useful for measuring that accumulated error.

Rotation Stress Test

```
def run_rotate_stress(self):
    slow_angular = self.config.get_variable("stress_test_rotation_speed") or 0.2
```

```

full_rotation = 2 * math.pi
...
while rclpy.ok():
    for rotation in range(10):
        time_for_rotation = full_rotation / slow_angular
        self.movement.cmd_vel_helper(0.0, slow_angular, time_for_rotation)

```

Runs in an infinite loop until `KeyboardInterrupt`. Each cycle completes 10 full rotations. The rotation speed comes from config (`stress_test_rotation_speed`), allowing tuning without code changes. `_print_elapsed_time` logs progress every 10 seconds so the operator knows the test is still running.

Circle Stress Test

```

def run_circle_stress(self, diameter: float):
    radius = diameter / 2.0
    circumference = 2 * math.pi * radius
    circle_time = circumference / linear_speed
    self.movement.cmd_vel_helper(linear_speed, angular_speed, circle_time)

```

For a circle of a given diameter, the required angular velocity is v / r (linear speed divided by radius). This value must be set in config as `angular_speed` before running — there’s no automatic calculation of the correct angular speed from the diameter. If the configured `angular_speed` doesn’t match `linear_speed / radius`, the robot traces an arc rather than a circle.

Observations

- The circle stress test silently produces wrong geometry if `angular_speed` isn’t set to `linear_speed / radius`. A `log_warn` when the computed ratio differs significantly from the configured value would help.
- Both stress tests catch `KeyboardInterrupt` and call `self.movement.stop()`. This is good practice — without it, a `Ctrl+C` mid-motion would leave the robot moving.
- `_print_elapsed_time` is a helper that returns `current_time` if it logged, `last_print_time` if not. The pattern is slightly awkward; a simple counter or `time.time()` modulo check would be cleaner.

ProcessApi: Subprocess Lifecycle Management

`ProcessApi` owns everything that involves spawning, monitoring, and killing OS-level processes. It has two distinct modes of operation: **async launch** (fire-and-forget background processes) and **sync execution** (blocking commands that return output). Both are used by `RobotController` for different purposes.

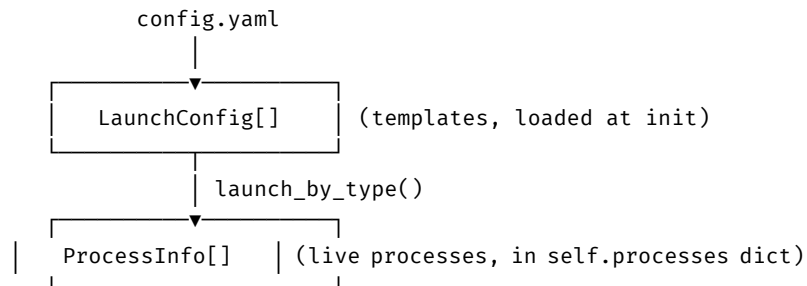
Data Structures

Two dataclasses carry all process state:

```
@dataclass
class ProcessInfo:
    process: subprocess.Popen
    command: str
    output: List[str]
    is_running: bool
    pid: int
    start_time: float
    log_file: Optional[str] = None
```

```
@dataclass
class LaunchConfig:
    launch_type: str
    command_template: str
    description: str
    default_params: Dict[str, str]
```

ProcessInfo is the runtime state of a running process. LaunchConfig is the static template loaded from YAML config.



Loading Launch Templates

ProcessApi.__init__ calls _load_launch_configs to convert the raw YAML dict into typed LaunchConfig objects:

```
def _load_launch_configs(self):
    templates = self.config.get_launch_templates() if self.config else {}
    self.launch_configs = {}
    for launch_type, template in templates.items():
        self.launch_configs[launch_type] = LaunchConfig(
            launch_type=launch_type,
            command_template=template.get("command", ""),
            description=template.get("description", ""),
```

```

        default_params=template.get("default_params", {})
    )

```

Async Launch: `launch_command`

The async path launches a process in a new session (`start_new_session=True`) so it survives terminal disconnection. A background thread captures all stdout to a log file and to the console:

```

proc = subprocess.Popen(
    command, shell=True,
    stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
    stdin=subprocess.DEVNULL,
    text=True, bufsize=1,
    start_new_session=True,
)

```

`start_new_session=True` creates a new process group, which matters for `kill_ros_process`: if the PID is the group leader, `SIGTERM` is sent to `-pgid` (the whole group), not just the parent.

A UUID is returned as the process handle. `RobotController` stores this UUID in `launch_process_ids` keyed by launch type.

Sync Execution: `run_command_sync`

Map saving and serialization need to run a command and wait for its result. `CommandConfig` bundles command, log name, and timeout:

```

@dataclass
class CommandConfig:
    command: str
    log_name: Optional[str]
    timeout: float

def run_command_sync(self, config: CommandConfig) → tuple[bool, str, Optional[str]]:
    result = subprocess.run(
        config.command, shell=True,
        capture_output=True, text=True,
        timeout=config.timeout
    )
    return (result.returncode == 0, result.stdout + result.stderr, log_file_path)

```

Returns (success, output, `log_file_path`) so callers can report both the outcome and where to find details.

Killing Processes

`kill_ros_process` handles the case where the target may be a process group leader — in which case `kill -TERM -PID` sends `SIGTERM` to the entire group:

```
target = f"-{pid}" if is_group_leader else str(pid)
subprocess.run(["kill", "-TERM", target], check=True, timeout=2)
```

If `SIGTERM` fails, it escalates to `SIGKILL`. This graceful-then-force pattern avoids leaving zombie processes.

Process Status Queries

`is_process_running` uses `poll()` to detect silent process death without blocking:

```
def is_process_running(self, process_id: str) → bool:
    proc_info = self.processes[process_id]
    proc_info.process.poll()
    if proc_info.process.returncode is not None:
        proc_info.is_running = False
        return False
    return True
```

`get_process_pid` provides the OS PID for external display:

```
def get_process_pid(self, process_id: str) → int | str:
    info = self.processes.get(process_id)
    return info.pid if info else "unknown"
```

Listing ROS and Launch Processes

`list_ros_processes` and `list_launch_processes` shell out to `ps aux` and `ps axo` respectively, filtering by ROS-related keywords. These are display-only utilities — they scan all OS processes, not just managed ones:

```
ps aux → filter ros/nav2/slam/rviz keywords → tabular output
ps axo pid,pgid,ppid → filter ros2 launch → parent-only display
```

Observations

- **Parameter formatting is complex.** `launch_by_type` has distinct code paths for `bl` commands, `ros2 run`, and launch files. This fragility could be replaced with a strategy object per command type.
- **`list_ros_processes` leaks shell knowledge.** The hardcoded keyword list `["ros2", "nav2", "slam", "rviz", ...]` will miss new components and is not tested.
- **Circular import via `string`.** `kill_ros_process` and `list_*` import `CommandResponse` inline to avoid circular imports. The fix is to move `CommandResponse` to its own module.

Announcement Contract

`announcement_contract.py` defines the shared data model for messages published to the `/announcement` topic. It is the contract between producers (behavior manager, voice input) and consumers (speech output node).

Why a Separate Contract Module

Multiple nodes in different files need to agree on the `/announcement` message structure. A shared module avoids each node reimplementing its own serialization logic and prevents the producer and consumer from drifting apart.

Priority Constants

The module establishes an ordered urgency scale:

```
PRIORITY_SAFETY = 0          # highest – obstacle, emergency
PRIORITY_STATUS = 1          # battery low, localization lost
PRIORITY_QUERY_REPLY = 2     # answer to a user question
PRIORITY_ACTION_ACK = 3      # "turning left"
PRIORITY_DISCOVERY = 4       # "I see a chair"
PRIORITY_CHITCHAT = 5        # lowest – greetings, idle commentary
```

Lower numbers = higher priority. The scale is not yet enforced by a priority queue in the speech output node, but is the intended semantic for future queueing logic.

Dual Import Strategy

The module uses `try/except ImportError` in two places to allow use outside a ROS2 environment:

```
try:
    from builtin_interfaces.msg import Time
except ImportError:
    @dataclass
    class Time:
        sec: int = 0
        nanosec: int = 0
```

And similarly for `AnnouncementMsg`. This lets unit tests import the module without a ROS2 installation. The stub classes provide just enough structure for pure-Python assertions.

The Announcement Dataclass

```
@dataclass(frozen=True)
class Announcement:
```

```

text: str
priority: int = PRIORITY_CHITCHAT
stamp: Time | None = None
source: str = "unknown"
dedup_key: str = ""

```

frozen=True makes instances hashable and immutable — safe to pass across ROS2 callbacks.

dedup_key is a forward-looking field intended for suppressing repeated identical announcements (e.g., a “battery low” warning that would otherwise fire every few seconds).

Serialization Paths

Producer	Consumer
Announcement	
→ to_payload() → JSON string	↔ from_payload() → Announcement
→ to_msg() → AnnouncementMsg	↔ from_msg() → Announcement

to_payload / from_payload use a plain JSON string over std_msgs/String. This allows nodes that don’t have the custom message type compiled to still participate via the JSON path.

from_payload is defensive: if the payload is not valid JSON, the whole string is treated as the announcement text. This lets legacy publishers still work.

Factory Helpers

```

def make_announcement_payload(text, *, priority, source, dedup_key) → str
def make_announcement_msg(text, *, priority, source, dedup_key, stamp) → AnnouncementMsg

```

Convenience wrappers so call sites don’t need to construct Announcement and immediately call .to_payload() or .to_msg().

Observations

- The dedup_key field is defined but nothing currently populates it with meaningful values. The speech output node does not suppress duplicates.
- Priority is not acted on anywhere. A future priority queue in SpeechOutputNode would need to buffer incoming announcements and sort by priority before speaking.

Movement Commands

movement_commands.py defines the command registry entries for all robot motion commands. It is the translation layer between user-facing CLI vocabulary and

RobotController method names.

What This Module Is and Isn't

This module contains *no logic*. It returns a dict of CommandDef objects. All motion implementation is in RobotController and MovementApi. The purpose of this file is to declare:

- What command names the CLI recognises
- What parameters each command expects
- Which RobotController method to call

The Registry

```
def build_movement_commands() → dict[str, cd.CommandDef]:
    return {
        "move.distance": CommandDef("move_distance", [ParameterDef("distance", float, True, ...)],),
        "move.time":      CommandDef("move_for_time", [ParameterDef("seconds", float, True, ...)],),
        "turn.time":      CommandDef("turn_for_time", [ParameterDef("seconds", float, True, ...)],),
        "turn.radians":   CommandDef("turn_radians", [ParameterDef("radians", float, True, ...)],),
        "turn.degrees":   CommandDef("turn_degrees", [ParameterDef("degrees", float, True, ...)],),
        "move.forward":   CommandDef("move_forward", [ParameterDef("meters", float, True, ...)],),
        "move.backward":  CommandDef("move_backward", [ParameterDef("meters", float, True, ...)],),
        "turn.clockwise": CommandDef("turn_clockwise", [ParameterDef("degrees", float, True, ...)],),
        "turn.counterclockwise": CommandDef("turn_counterclockwise", [ParameterDef("degrees", float, True, ...)],),
    }
```

Command Name Convention

Keys use group.subcommand dot notation. CommandDispatcher treats the dot as a separator between the command group and the subcommand variant. SimpleParser maps user input "move forward 1.5" to the key "move.forward" by joining the first two resolved tokens with a dot.

Semantic Duplication

Notice that move.distance / move.forward / move.backward all ultimately call move_dist on MovementApi, with move.forward forcing positive and move.backward forcing negative distance. This redundancy is intentional: it lets users speak in whichever vocabulary feels natural (move distance -1.0 vs move backward 1.0).

Observations

- All parameters are required=True with default=None. There are no optional motion parameters. Adding defaults (e.g., a default distance of 1.0 metre) would make one-word commands like move forward work.

- `turn.radians` and `turn.degrees` both exist, which is good for users who think in either unit. They call different `RobotController` methods which in turn call different `MovementApi` methods (`turn_amount` for radians, `turn_degrees` which converts and calls `turn_amount`).

Launch Commands

`launch_commands.py` defines the command registry entries for managing ROS2 launch processes. These commands expose `ProcessApi`'s lifecycle management through the CLI.

The Registry

```
"launch.list": CommandDef("launch_list", [], ... )
"launch.info": CommandDef("launch_info", [ParameterDef("launch_type", str, True, ... )], ... )
"launch.start": CommandDef("launch_start", [
    ParameterDef("launch_type", str, True, None, "Launch type (nav, slam, map)"),
    ParameterDef("use_sim_time", bool, False, None, "Use simulation time"),
    ParameterDef("map", str, False, None, "Map filename"),
    ParameterDef("map_name", str, False, None, "Map name for map launch type"),
], ... )
"launch.stop": CommandDef("launch_stop", [ParameterDef("launch_type", str, True, ... )], ... )
```

Parameter Design for `launch.start`

`launch.start` has four parameters but only `launch_type` is required. The optional parameters (`use_sim_time`, `map`, `map_name`) are passed as `**kwargs` through `RobotController.launch_start` into `ProcessApi.launch_by_type`, which adds non-None values to the command template.

This means the CLI user writes:

```
launch start nav --map basement
```

and `ProcessApi` resolves `basement` to the full YAML path under `maps_dir` and appends `map:=/full/path/basement.yaml` to the `ros2 launch` command.

What `launch.list` Shows

`launch.list` calls `RobotController.launch_list`, which queries `ProcessApi.get_available_launch_types()` and formats a table:

NAME	DESCRIPTION

nav	Nav2 navigation stack
slam	SLAM Toolbox online async
map	Map server

These names come directly from the `launch_templates` keys in `config.yaml`. If the config has no templates, this command returns “No launch templates available.”

Observations

- `launch.info` returns the raw command template, which exposes implementation details (full `ros2 launch` command strings). This is useful for debugging but might be surprising to end users.
- There is no `launch.status` command to see which launch types are currently running. `robot status` includes this information but it’s buried in a larger status dump.
- `map_name` and `map` are distinct parameters that behave differently in `ProcessApi.launch_by_type` — `map` is a map file for nav, while `map_name` is for the map server launch type. This is a confusing API that could be unified.

System Commands

`system_commands.py` defines the command registry entries for configuration management, scripted movement patterns, and system-level process inspection. It bundles four conceptually distinct groups: `config`, `script`, `system`, and implicitly exposes `map` operations via the navigation group.

Groups Covered

`config`

<code>config.set</code>	<code>name value</code>	→ <code>RobotController.set_variable</code>
<code>config.get</code>	<code>name</code>	→ <code>RobotController.get_variable</code>
<code>config.list</code>		→ <code>RobotController.get_all_variables</code>

Exposes `ConfigManager` through the CLI. `config.list` filters out `launch_templates` from the output (it’s too verbose for a terminal display).

`script`

<code>script.list</code>		→ lists available scripts
<code>script.square</code>	<code>meters</code>	→ <code>CalibrationApi.run_square_pattern</code>
<code>script.rotate_stress</code>		→ <code>CalibrationApi.run_rotate_stress</code>
<code>script.circle_stress</code>	<code>diameter</code>	→ <code>CalibrationApi.run_circle_stress</code>

These are the only commands that directly invoke multi-step, long-running motion sequences. They block the CLI until the pattern completes or the user interrupts with `Ctrl+C`.

system

`system.topics` → list active /topics via `MovementApi.get_topic_names_and_types`
`system.ps` → list all ROS-related OS processes (`ps aux` filtered)
`system.kill pid` → kill process by PID (group kill if `PID == PGID`)
`system.launches` → list ros2 launch parent processes (filtered from `ps`)

`system.kill` is noteworthy: `RobotController.kill_ros_process` first checks whether the target PID is a process group leader by comparing `PID == PGID`. If it is, it sends `SIGTERM` to the whole process group (`kill -TERM -PID`). This is the correct way to shut down a `ros2 launch` invocation — killing just the launcher process leaves all its children orphaned.

Observations

- `system.topics` uses `self.movement.get_topic_names_and_types()`, which creates a `MovementApi` node on first call. This has the side effect of initialising a full ROS2 subscriber set just to list topics. A lightweight dedicated node or `ros2 topic list` subprocess call would be cleaner.
- `script.rotate_stress` has no timeout parameter — it runs until `Ctrl+C`. For remote operation this is hazardous; adding an optional `--max-seconds` parameter would be safer.
- `config.set` and `config.get` are in `system_commands.py` but the config group name is distinct from `system`. Future refactoring could split config into its own file.

RobotController: Orchestrating Robot Operations

`RobotController` is the business logic layer. It sits between the command interfaces (CLI, REST) and the ROS2 API nodes (`MovementApi`, `ProcessApi`, `CalibrationApi`, `IntentApi`). Its job is to coordinate those APIs and return `CommandResponse` objects that callers can format however they like.

Uniform Return Type

Every public method returns a `CommandResponse`:

```
@dataclass
class CommandResponse:
    success: bool
    message: str
    data: dict | None = None
```

This is a deliberate design decision. Without it, callers need to handle `None`, exceptions, tuples, and strings. With it, the CLI can always check `result.success` and print `result.message`.

Lazy API Initialization

ROS2 nodes are expensive to construct — they register with the DDS middleware and allocate system resources. `RobotController` defers construction until first use via properties:

```
@property
def movement(self) → MovementApi:
    if self.movement_node is None:
        self.movement_node = MovementApi(self.config)
    return self.movement_node
```

The backing variables (`movement_node`, `calibration_node`, `process_node`, `intent_node`, `speech_node`) start as `None`. When `get_robot_status` inspects them, it checks for `None` directly — a non-`None` node means it was constructed and is therefore “available”.

```
nodes_status = {
    "movement_api": "running" if self.movement_node else "not available",
    "calibration_api": "running" if self.calibration_node else "not available",
    "process_api": "running" if self.process_node else "not available",
}
```

Launch Process Tracking

`RobotController.__init__` pre-populates `launch_process_ids` with `None` values for every known launch type:

```
self.launch_process_ids = dict.fromkeys(self.config.get_launch_templates().keys())
```

This means iteration over `launch_process_ids` always covers all configured launch types, whether running or not — no surprises when listing status.

```
launch_process_ids = {
    "nav": "3f2a- ..." # running — stores UUID from ProcessApi
    "slam": None          # not running
    "map": None           # not running
}
```

Launch Start and Stop

`launch_start` checks for a conflict (already running), then delegates to `ProcessApi.launch_by_type` and stores the returned UUID:

```
def launch_start(self, launch_type: str, **kwargs) → CommandResponse:
    if self.is_launch_running(launch_type):
        return CommandResponse(False, f"{launch_type} is already running, stop it first")
    try:
        process_id = self.process.launch_by_type(launch_type, **kwargs)
        self.launch_process_ids[launch_type] = process_id
```



```

        return CommandResponse(True, f"Started {launch_type}", {"process_id": process_id})
    except ValueError as e:
        return CommandResponse(False, str(e))

```

launch_stop mirrors this: look up the UUID, verify it is actually running, kill it, clear the UUID.

Map Operations

Map save and serialize share a common pattern: build a shell command, wrap it in a CommandConfig, run it synchronously, and interpret the result:

```

def map_path(self):
    map_name = self.config.get_variable("map_name")
    if not map_name:
        return None, None, None
    self.config.ensure_subdirs()
    maps_dir = self.config.get_maps_dir()
    return map_name, maps_dir, maps_dir / map_name

def run_map_command(self, config: CommandConfig, success_msg: str, error_prefix: str) → CommandResponse:
    success, output, log_file = self.process.run_command_sync(config)
    if success:
        return CommandResponse(True, success_msg + (f"\nLog: {log_file}" if log_file else ""))
    ...

```

map_path is extracted because both map_save and map_serialize need the same three values: name, directory, and full path. Without the helper, that logic would be duplicated.

Speech Output

speak_text publishes text to /announcement via SpeechApi, which SpeechOutputNode picks up and converts to audio. The speech property follows the same lazy init pattern as other APIs:

```

@property
def speech(self) → SpeechApi:
    if self.speech_node is None:
        self.speech_node = SpeechApi(self.config)
        time.sleep(0.5)
    return self.speech_node

def speak_text(self, text: str) → CommandResponse:
    self.speech.speak(text)
    return CommandResponse(True, f"Speaking: {text!r}")

```

The 0.5s sleep on first construction serves the same DDS discovery purpose as

the sleep on `intent` — allows the publisher to be seen by subscribers before the first message.

Intent Publishing

The `publish_intent` methods dispatch named intents to the behavior manager via `IntentApi`:

```
def publish_intent(self, name: str, slots: dict) → CommandResponse:
    self.intent.publish(name, "cli", slots)
    return CommandResponse(True, f"Intent published: {name}")
```

The `intent` property has a 0.5-second sleep after first construction to allow DDS publisher-subscriber discovery before the first publish. This is a known ROS2 startup race condition.

Delegating to ProcessApi

Process management methods delegate entirely to `ProcessApi`:

```
def list_ros_processes(self) → CommandResponse:
    return self.process.list_ros_processes()

def kill_ros_process(self, pid: int) → CommandResponse:
    return self.process.kill_ros_process(pid)
```

This keeps `RobotController` free of subprocess and `ps` parsing logic.

Survey Start

`start_survey` uses `SurveyApi` to call the `/survey/start` Trigger service on `SpinSurveyNode`. The `survey` property follows the same lazy init pattern as other APIs:

```
@property
def survey(self) → SurveyApi:
    if self.survey_node is None:
        self.survey_node = SurveyApi()
    return self.survey_node

def start_survey(self) → CommandResponse:
    ok, msg = self.survey.start()
    return CommandResponse(ok, msg)
```

No DDS discovery sleep is needed here because `SurveyApi.start()` calls `wait_for_service()` internally before sending the request.

Observations

- **turn_degrees breaks the pattern.** It returns `self.movement.turn_degrees(degrees)` directly instead of a `CommandResponse`. `MovementApi.turn_degrees` returns `None`, so callers get `None` back — a latent bug.
- **No error handling on movement calls.** Methods like `move_distance` call `self.movement.move_dist` without `try/except`. A ROS2 exception (e.g., node not initialized) would propagate unhandled to the CLI.
- **list_maps complexity.** The file-grouping logic (`stem` → `extensions`) is 20 lines that could be a helper. It also conflates map files and any other file in the directory.

CommandDispatcher: Command Registry and Execution

`CommandDispatcher` is the single entry point for all command execution. The CLI, REST API, and any future interface call `dispatcher.execute(name, params)` or `dispatcher.dispatch_text(text)` and get back a `CommandResponse`. The dispatcher handles parameter validation, type coercion, and method dispatch — callers never touch `RobotController` methods directly.

Registry as a Dict

The command registry is a plain `dict[str, CommandDef]` built at construction time:

```
def _build_command_registry(self) → dict[str, cd.CommandDef]:
    commands = {}
    commands.update(mov_cmd.build_movement_commands())
    commands.update(ctrl_cmd.build_control_commands())
    commands.update(nav_cmd.build_navigation_commands())
    commands.update(lch_cmd.build_launch_commands())
    commands.update(sys_cmd.build_system_commands())
    commands.update(surv_cmd.build_survey_commands())
    return commands
```

Each `build_*` function returns a dict of `"group.command" → CommandDef`. Intent commands and semantic commands were in this registry prior to F15/T07; they are now handled entirely by `dispatch_text` routing. The flat merge means duplicate names from different modules would silently shadow each other. Survey commands were added 2026-05-14 when `SpinSurvey` moved from `dome_vision` to `dome_control`.

The Execute Path

```
dispatcher.execute("move.forward", {"meters": "1.5"})
```

```

|
├─ look up CommandDef in registry
├─ _validate_parameters → coerce "1.5" → 1.5 (float)
├─ getattr(robot_controller, "move_forward")
└─ method(meters=1.5) → CommandResponse

```

dispatch_text: Text-to-Command Routing

dispatch_text(text) is the entry point for free-form text input (CLI, voice transcripts):

```

def dispatch_text(self, text: str) → rc.CommandResponse:
    tokens = text.strip().split()
    command = resolve_keyword(tokens[0])
    ...
    intent_name = BEHAVIOR_COMMANDS.get(command_name)
    if intent_name is not None:
        reply = self.publish_intent(intent_name, slots)
        msg = reply if reply is not None else f"Intent published: {intent_name}"
        return rc.CommandResponse(True, msg)
    ...
    return self.execute(command_name, params)

```

Two routing paths:

- **Behavior path** — command name is in BEHAVIOR_COMMANDS → publish to /intent via IntentPublisher, return immediately.
- **Direct path** — everything else → look up CommandDef, validate params, call RobotController method.

BEHAVIOR_COMMANDS

Maps CLI command names to intent names:

```

BEHAVIOR_COMMANDS: dict[str, str] = {
    "intent.stop": "stop",
    "intent.explore": "explore",
    "intent.describe_scene": "describe_scene",
    "intent.count_objects": "count_objects",
    "intent.list_objects": "list_objects",
    "scene.describe": "describe_scene",
    "scene.count": "count_objects",
    "scene.objects": "list_objects",
}

```

scene.* forms are preferred interactive vocabulary; intent.* forms kept for scripting. publish_intent returns str | None — if IntentApi waited for a

reply (query intents), the reply text comes back and replaces the default “Intent published” message in the `CommandResponse`.

Abbreviation Resolution

`ABBREV_TO_FULL` and `FULL_NAMES` are module-level constants. `resolve_keyword` expands short tokens:

```
ABBREV_TO_FULL = {"m": "move", "fwd": "forward", "stp": "stop", ...}
FULL_NAMES = set(ABBREV_TO_FULL.values())
```

```
def resolve_keyword(word: str) → str:
    if word in FULL_NAMES:
        return word
    return ABBREV_TO_FULL.get(word, word)
```

Unknown tokens pass through unchanged; errors surface at execution time.

Subcommand Detection

After resolving the first token, the parser checks whether the second token forms a compound command:

```
second = resolve_keyword(tokens[1])
candidate = f"{command}.{second}"
in_registry = candidate in self.commands or candidate in BEHAVIOR_COMMANDS
if second in FULL_NAMES or second != tokens[1] or in_registry:
    command_name = candidate # compound: "move.forward"
    args = [parse_value(t) for t in tokens[2:]]
else:
    command_name = command # simple: "stop"
    args = [parse_value(t) for t in tokens[1:]]
```

The `in_registry` check ensures scene describe routes correctly even though “describe” is not in `FULL_NAMES`.

IntentPublisher Injection

```
def __init__(self, robot_controller, intent_publisher=None):
    ...
    self.intent_publisher = intent_publisher
```

In production, leave `intent_publisher=None` — `publish_intent` creates an `IntentPublisher` lazily. In tests, pass `IntentPublisher(publish_fn=published.append)` to capture published payloads without ROS2.

Parameter Validation and Coercion

`_convert_parameter_value` handles booleans specially:

```

if param_def.param_type == bool:
    if isinstance(value, str):
        return value.lower() in ("true", "1", "yes", "on")
    return bool(value)

```

Python's `bool("false")` is `True`, so string booleans need explicit mapping.

Observations

- **Silent shadowing.** Duplicate keys across `build_*` functions are silently overwritten.
- **Method lookup at runtime.** A stale `method_name` in a `CommandDef` fails at call time, not at registry build time.

SpeechApi — Announcement Publisher

Purpose

`SpeechApi` is the CLI-side ROS2 node for publishing speech to the `/announcement` topic. It wraps `Announcement` dataclass construction and publishing into a single `speak()` call, giving `RobotController` a clean interface for text output without touching ROS2 message types directly.

Design

`SpeechApi` extends `BaseApi` (the shared ROS2 node base class). Like all API classes, it is instantiated lazily by `RobotController` on first use:

```

@property
def speech(self) → SpeechApi:
    if self.speech_node is None:
        self.speech_node = SpeechApi(self.config)
    return self.speech_node

```

The publisher is created in `__init__` but `AnnouncementMsg` is imported lazily inside `__init__` to avoid pulling in ROS2 message types at module load:

```

def __init__(self, config_manager: cm.ConfigManager = None):
    super().__init__("speech_api", config_manager)
    from dome_control.announcement_contract import AnnouncementMsg
    self.announcement_pub = self.create_publisher(AnnouncementMsg, "/announcement", 10)

```

The `speak()` Method

```

def speak(self, text: str, priority: int = PRIORITY_CHITCHAT) → None:
    ann = Announcement(text=text, priority=priority, source="cli")
    self.announcement_pub.publish(ann.to_msg())
    self.log_info(f"Speak published: {text!r}")

```

source="cli" tags all CLI-originated speech. Priority defaults to `PRIORITY_CHITCHAT` — the lowest tier, meaning `SpeechOutputNode` may defer or drop it if higher-priority messages are queued.

Observations

- Thin wrapper with one method. Could be a free function if `BaseApi` weren't needed for the publisher lifecycle.
- source is hardcoded to "cli". If other non-node callers use this class, the source tag would be wrong.
- `PRIORITY_CHITCHAT` default means CLI speech can be preempted by robot-generated announcements. This is intentional — robot operational messages take precedence.

SimpleCLI

`simple_cli.py` is the top-level entry point for the robot control CLI. It assembles the full stack — config, controller, dispatcher — and provides both an interactive REPL and a non-interactive single-command mode.

Assembly at Startup

```
class SimpleCLI:
    def __init__(self):
        default_cfg = str(Path.home() / ".control" / "config.yaml")
        config_path = os.environ.get("CONTROL_CONFIG", default_cfg)
        self.config_manager = cm.ConfigManager.create(config_path)
        self.robot_controller = rc.RobotController(self.config_manager)
        self.intent_publisher = IntentPublisher()
        self.intent_publisher.get_api() # warm up ROS2 node + DDS discovery at startup
        self.dispatcher = cd.CommandDispatcher(self.robot_controller, self.intent_publisher)
```

Construction order: `ConfigManager` → `RobotController` → `IntentPublisher` (warmed up) → `CommandDispatcher`. `CONTROL_CONFIG` env var overrides the default config path.

`IntentPublisher` is constructed and its `IntentApi` ROS2 node is created eagerly at startup (the `.get_api()` call). This ensures DDS publisher-subscriber discovery happens before the first intent is published, avoiding the race condition where early commands are dropped.

`SimpleCommandParser` was removed in F15/T05. All text parsing now goes through `CommandDispatcher.dispatch_text`.

Two Modes

```
def main():
    cli = SimpleCLI()
    if len(sys.argv) > 1:
        cli.execute_command(" ".join(sys.argv[1:]))
    else:
        cli.repl()
```

Non-interactive: `ros2 run dome_control run move forward 1.0` — executes one command and exits. Interactive REPL: `prompt_toolkit` with persistent history in `~/.control/prompt_history.txt`. Commands are also timestamped to `~/.control/command_history.txt`.

execute_command

```
def execute_command(self, input_text: str) → None:
    tokens = input_text.strip().split()
    first = tokens[0].lower()

    if first in ("help", "hlp"):
        subcommand = resolve_keyword(tokens[1]) if len(tokens) > 1 else None
        arguments = [resolve_keyword(t) for t in tokens[2:]]
        self.handle_help(subcommand, arguments)
        return

    if first in ("exit", "quit", "q", "x"):
        ...

    result = self.dispatcher.dispatch_text(input_text)
```

Three paths: help handling (in-CLI), exit handling (in-CLI), everything else goes to `dispatch_text`. Help and exit are intercepted before dispatch because they require CLI-level state (`self.running`, `self.dispatcher.commands`).

Help System

```
help                → show_common_commands()
help commands       → show_all_commands()
help <group>         → show_subcommand_suggestions()
help <group> <subcommand> → load_help_file() → docs/<group>.<sub>.txt
```

`show_all_commands` dumps the full `dispatcher.commands` registry alphabetically, including behavior commands (intent/scene group). Detailed help files load from `docs/` relative to the package source root.

Error Feedback

When an unknown command is entered:

```
if "Unknown command" in result.message and "." not in tokens[0]:
    self.show_subcommand_suggestions(tokens[0], "Did you mean one of these?")
```

Lists all <group>.* commands if the token looks like a group name.

Observations

- `log_command` writes to file on every command with no buffering or rotation — history file grows indefinitely.
- `load_help_file` traverses three levels up from `__file__` to find `docs/`. Brittle to layout changes.
- Empty input is silently ignored in both REPL and non-interactive mode.

IntentParser: Pure Intent Parsing Logic

`behavior_manager.py` holds `IntentParser` — pure logic for parsing and validating intents with no ROS2 dependency. The name `BehaviorManager` was retired in F15/T01; the class is now `IntentParser` to reflect its single responsibility.

The ROS2 node layer (`BehaviorManagerNode`) handles subscriptions and callbacks; `IntentParser` handles only the JSON parsing semantics. Tests for intent parsing run without a ROS2 context.

The Intent Dataclass

```
@dataclass
class Intent:
    name: str
    source: str
    slots: dict
```

An intent has a name (what to do), a source (who asked), and slots (parameters). The slots dict is open-ended — different intents carry different slot keys. Validation of slot contents is left to the downstream handler.

Parsing with Strict Validation

`parse_intent` takes a raw JSON string — as it arrives over a ROS2 topic — and returns a typed `Intent` or raises `ValueError`:

```
def parse_intent(self, payload: str) → Intent:
    try:
        raw = json.loads(payload)
    except json.JSONDecodeError as exc:
```

```

        raise ValueError(f"Invalid intent JSON: {exc.msg}") from exc

    if not isinstance(raw, dict):
        raise ValueError("Intent payload must be a JSON object")

    name = raw.get("name")
    if not isinstance(name, str) or not name:
        raise ValueError("Intent field 'name' must be a non-empty string")

    source = raw.get("source", "unknown")
    slots = raw.get("slots", {})
    if not isinstance(slots, dict):
        raise ValueError("Intent field 'slots' must be an object")

    return Intent(name=name, source=source, slots=slots)

```

Each check raises with a descriptive message. The caller (the ROS2 node) can log the error and discard the message without crashing.

Announcement Helpers

Two helpers are re-exported from this module:

```

from dome_control.announcement_contract import (
    make_announcement_msg,
    make_announcement_payload,
)

```

The actual implementation lives in `announcement_contract.py` to avoid a circular dependency with the ROS2 message type.

Observations

- **No slot validation.** `parse_intent` validates structure but not slot contents. An intent `{"name": "count_objects", "slots": {}}` (missing `object_type`) will parse successfully and fail later.
- **source default is “unknown”.** Could be required rather than silently defaulted.
- **Single method class.** `IntentParser` has one public method. The class form is kept for future expansion (e.g., intent history, rate limiting).

BehaviorManagerNode — Intent Router

Purpose

`BehaviorManagerNode` is the central ROS2 node that receives structured intents from `/intent` and dispatches them to domain-specific behavior handlers. It also

caches sensor data from `/oak/detections` and loads vision class profiles for label resolution.

Initialization: Handlers and Vision Config

At startup the node wires two behavior handlers and optionally loads class profiles from `dome_vision`:

```
rc = RobotController(ConfigManager.create(DEFAULT_CONFIG))
self.handlers = [
    MotionBehavior(rc),
    PerceptionBehavior(self),
]
```

Class profiles are loaded from a YAML file whose path is settable via a ROS2 parameter, defaulting to the `dome_vision` config location:

```
self.declare_parameter("class_profiles_path", DEFAULT_VISION_CONFIG)
vision_config_path = self.get_parameter("class_profiles_path").get_parameter_value().string_value
self.profiles: dict = {}
self.label_map: dict = {}
try:
    from dome_vision.class_profiles import build_label_map, load_class_profiles
    self.profiles = load_class_profiles(vision_config_path)
    self.label_map = build_label_map(self.profiles)
except Exception as exc:
    self.get_logger().warn(f"Could not load class profiles from {vision_config_path}: {exc}")
```

The `try/except` means the node runs without `dome_vision` installed — perception commands degrade to raw class IDs rather than failing entirely.

Subscriptions and Publishers

```
self.intent_sub = self.create_subscription(
    String, "/intent", self.on_intent, 10
)
self.detections_sub = self.create_subscription(
    Detection2DArray, "/oak/detections", self.on_detections, 10
)
self.latest_detections: Detection2DArray | None = None
self.announcement_pub = self.create_publisher(
    AnnouncementMsg, "/announcement", 10
)
self.describe_scene_client = self.create_client(Trigger, "/describe_scene")
```

`/oak/detections` is optional — the subscription exists whether or not oak is running. When no oak node is present, `latest_detections` stays `None` and `PerceptionBehavior.report_detections()` returns “No objects detected.” cleanly.

Detection Cache

A one-liner callback keeps the latest frame:

```
def on_detections(self, msg: Detection2DArray) → None:
    self.latest_detections = msg
```

Intent Dispatch

Intents are parsed from JSON, matched against handlers, and executed. `get_help` is handled inline because it requires no domain knowledge:

```
def on_intent(self, msg: String) → None:
    try:
        intent = self.parser.parse_intent(msg.data)
    except ValueError as exc:
        self.get_logger().warn(str(exc))
        return

    if intent.name == "get_help":
        ... announce command list ...
        return

    for handler in self.handlers:
        if handler.handles(intent.name):
            handler.execute(intent, self)
            return

    self.get_logger().warn(f"Unhandled intent: {intent.name}")
```

flowchart TD

```
A[/intent topic] --> B[parse_intent]
B -- ValueError --> C[log warn]
B -- Intent --> D{get_help?}
D -- yes --> E[announce command list]
D -- no --> F[MotionBehavior.handles?]
F -- yes --> G[MotionBehavior.execute]
F -- no --> H[PerceptionBehavior.handles?]
H -- yes --> I[PerceptionBehavior.execute]
H -- no --> J[log unhandled]
```

Observations

- `get_help` handled inline is a mild violation of the handler pattern. A `HelpBehavior` would be consistent.
- `latest_detections` is mutable shared state updated by a ROS2 callback and read by a behavior. In a multi-threaded executor this is a race; the default single-threaded executor makes it safe.

- `profiles` and `label_map` are set on the node so `PerceptionBehavior` can access them via `self.node`. This is a form of implicit coupling — `PerceptionBehavior` must know the node has these attributes. An explicit dependency injection would be cleaner.
- The `dome_vision` import is inside a `try/except` at init time, not deferred. If `dome_vision` is importable but the YAML is malformed, the node still starts with empty profiles rather than crashing.

SpeechOutputNode

`speech_output_node.py` subscribes to `/announcement` and converts text to speech using Piper TTS and ALSA playback. It is the voice of the robot.

Pipeline

```
/announcement (AnnouncementMsg)
  ↓ on_announcement
  SpeechOutputNode
    ↓ synthesize_to_wav (Piper)
    /tmp/speech-output-XXXX.wav
      ↓ apply_wav_gain (optional)
      /tmp/speech-output-YYYY.wav
        ↓ play_wav (aplay)
        speaker output
          ↓ cleanup temp files
```

Piper TTS Integration

```
def synthesize_to_wav(text, wav_path, piper_bin, model_path, length_scale):
    cmd = [piper_bin, "--model", model_path, "--output_file", wav_path,
           "--length_scale", str(length_scale), "--quiet"]
    subprocess.run(cmd, input=text.encode("utf-8"), check=True, capture_output=True)
```

Piper is invoked as an external subprocess. Text is passed on stdin; the WAV file is written to a temp path. `--quiet` suppresses Piper's progress output. `length_scale` controls speech speed (`>1.0` = slower, `<1.0` = faster); the default is 1.25 — slightly slower than natural speed for clarity in a robot environment.

Volume Control via PCM Gain

Piper has no built-in volume control. Instead, the node applies a software gain to the raw PCM samples:

```
def _scale_pcm_frames(frames: bytes, sample_width: int, gain: float) → bytes:
    max_value = (1 << (sample_width * 8 - 1)) - 1
    min_value = -(1 << (sample_width * 8 - 1))
```

```

for offset in range(0, len(frames), sample_width):
    sample = struct.unpack_from(fmt, frames, offset)[0]
    adjusted = max(min(int(sample * gain), max_value), min_value)
    scaled.extend(struct.pack(fmt, adjusted))
return bytes(scaled)

```

The default `SPEECH_GAIN = 0.35` attenuates the output to 35% of full scale. This is a workaround for Piper models that generate audio at a high peak amplitude that would clip on the robot's speaker. Gain is applied in pure Python with `struct.pack/unpack` — not fast, but audio synthesis time (seconds) dominates over gain application time (milliseconds).

Environment Variable Configuration

All tunable parameters come from environment variables with sensible defaults:

Variable	Default	Purpose
PIPER_BIN	"piper"	Path to Piper binary
PIPER_MODEL_PATH	""	Required — raises <code>RuntimeError</code> if empty
SPEECH_ALSA_DEVICE	""	ALSA device string (e.g., "hw:1,0")
SPEECH_TMP_DIR	<code>tempfile.gettempdir()</code>	Directory for temp WAV files
SPEECH_GAIN	"0.35"	PCM amplitude multiplier
PIPER_LENGTH_SCALE	"1.25"	Speech rate (>1 = slower)

`PIPER_MODEL_PATH` is the only genuinely required variable — `synthesize_to_wav` raises `RuntimeError` at call time if it's empty.

Temp File Cleanup

```

def speak_text(self, text: str) → None:
    wav_path = self._make_wav_path()
    gained_path = None
    try:
        synthesize_to_wav(...)
        if self.speech_gain != 1.0:
            gained_path = self._make_wav_path()
            apply_wav_gain(wav_path, gained_path, self.speech_gain)
            playback_path = gained_path
        play_wav(playback_path, ...)
    finally:

```

```

    for path in (wav_path, gained_path):
        if path:
            try: os.remove(path)
            except FileNotFoundError: pass

```

Both temp files (original and gained) are cleaned up in finally. The `FileNotFoundError` suppression handles the case where synthesis failed before the file was created.

Observations

- `speak_text` blocks until Piper finishes synthesis and `aplay` finishes playback. During this time, /announcement messages queue in the ROS2 subscriber buffer (depth=10). If multiple announcements arrive while the robot is speaking, they will be spoken sequentially. There is no skip, interrupt, or priority-based preemption.
- `_scale_pcm_frames` only supports sample widths 1, 2, and 4 bytes. Piper always outputs 16-bit (2-byte) WAV, so this is not a practical limitation.
- `parse_speech_gain` and `parse_length_scale` raise `ValueError` with descriptive messages. These are called in `__init__`, so a bad env var kills the node at startup with a clear error — good fail-fast behaviour.

MotionBehavior

`dome_control/behaviors/motion_behavior.py` handles motion-domain intents. It has no ROS2 dependency — all execution goes through `RobotController`, except for `get_status` which needs to publish an announcement via the node.

Interface

```

MOTION_INTENTS = {"stop", "explore", "drive_square", "turn_right", "turn_left", "get_status"}
TURN_DEGREES = 90

```

```

class MotionBehavior:
    def __init__(self, robot_controller): ...
    def handles(self, intent_name: str) → bool: ...
    def execute(self, intent: Intent, node=None) → None: ...

```

`handles` returns True for any name in `MOTION_INTENTS`. `execute` dispatches on `intent.name`:

```

def execute(self, intent: Intent, node=None) → None:
    if intent.name == "stop":
        self.rc.stop_robot()
    elif intent.name == "drive_square":
        meters = float(intent.slots.get("meters", 1.0))
        self.rc.script_square(meters)

```

```

elif intent.name == "explore":
    pass # not yet implemented
elif intent.name == "turn_right":
    self.rc.turn_clockwise(TURN_DEGREES)
elif intent.name == "turn_left":
    self.rc.turn_counterclockwise(TURN_DEGREES)
elif intent.name == "get_status" and node is not None:
    result = self.rc.get_robot_status()
    ...
    node.announcement_pub.publish(msg)

```

Turn Commands

`turn_right` and `turn_left` delegate to `RobotController.turn_clockwise / turn_counterclockwise`, which convert degrees to radians and call `MovementApi.turn_degrees`. The constant `TURN_DEGREES = 90` is defined at module level so it can be adjusted without searching inside the dispatch logic.

Sign convention follows ROS standard: counterclockwise is positive angular velocity, so `turn_counterclockwise` passes a positive angle and `turn_clockwise` negates it.

Status Announcement

`get_status` is the one intent that needs the node reference, to publish a spoken response:

```

elif intent.name == "get_status" and node is not None:
    result = self.rc.get_robot_status()
    status = result.data.get("status", {}) if result.data else {}
    speeds = status.get("speeds", {})
    text = (
        f"linear speed {speeds.get('linear', 'unknown')}, "
        f"angular speed {speeds.get('angular', 'unknown')}"
    )
    msg = make_announcement_msg(text, priority=PRIORITY_QUERY_REPLY, source="motion_behavior")
    node.announcement_pub.publish(msg)

```

The guard and `node is not None` means the intent is silently ignored if called without a node (e.g. from tests or CLI). This keeps the handler safe without requiring a real publisher in every context.

Slot Handling

`drive_square` reads `slots["meters"]` with a default of `1.0`. All other current motion intents are parameterless.

Observations

- `explore` is a no-op. The implementation slot exists so the intent routes correctly without an “Unhandled intent” warning.
- `cmd_vel_helper` blocks the `behavior_manager` callback thread for the full turn duration. No other intents are processed during a turn.
- Adding a new motion intent requires: adding the name to `MOTION_INTENTS` and adding an `elif` branch in `execute`.

PerceptionBehavior — Perception Intent Handler

Purpose

`PerceptionBehavior` handles perception-domain intents routed from `BehaviorManagerNode`. It either calls a ROS2 service (`describe_scene`) or reads cached sensor data (`list_objects`) and publishes results to `/announcement`.

Intent Routing

Three intents are handled:

```
PERCEPTION_INTENTS = {"describe_scene", "count_objects", "list_objects"}
```

```
def execute(self, intent: Intent, node=None) → None:
    if intent.name == "describe_scene":
        self.call_describe_scene()
    elif intent.name == "list_objects":
        self.report_detections()
    elif intent.name == "count_objects":
        self.node.get_logger().warn("count_objects intent not yet implemented")
```

`describe_scene`: Unconditional Async Service Call

The `/describe_scene` service (provided by `describe_scene_stub` or a real vision node) returns a string description via `Trigger`. The call is async — the result arrives in `on_describe_scene_done`.

A key design choice: the node calls the service even if `service_is_ready()` returns `False`. A warning is logged, but the call proceeds. This avoids silent no-ops when the service is slow to register during startup:

```
def call_describe_scene(self) → None:
    client = self.node.describe_scene_client
    if not client.service_is_ready():
        self.node.get_logger().warn("/describe_scene not ready – attempting call anyway")
    from std_srvs.srv import Trigger # lazy – avoids rclpy at import time
```

```

future = self.node.describe_scene_client.call_async(Trigger.Request())
future.add_done_callback(self.on_describe_scene_done)

```

The tradeoff: if the service never comes up, the future never resolves and the callback is never called. That is acceptable — the system degrades silently rather than failing loudly.

The callback handles both failure modes — an exception (service timeout or crash) and a successful call that returns `success=False`:

```

def on_describe_scene_done(self, future) → None:
    try:
        response = future.result()
    except Exception as exc:
        self.node.get_logger().error(f"/describe_scene call failed: {exc}")
        return

    if not response.success:
        self.node.get_logger().warn(
            f"/describe_scene returned failure: {response.message}"
        )
        return

    self.publish_announcement(response.message)

```

list_objects: Reading Cached Detections with Label Resolution

BehaviorManagerNode caches the latest `Detection2DArray` from `/oak/detections`. `report_detections` reads that cache, picks the highest-scoring hypothesis per detection, and formats a sentence.

A significant feature is label resolution. Raw class IDs from the vision model (e.g. `"person_0"`) are mapped through two layers: `label_map` canonicalizes the raw ID, and `profiles` provides a human-readable `display_name`:

```

def report_detections(self) → None:
    detections = getattr(self.node, "latest_detections", None)
    if detections is None or not detections.detections:
        self.publish_announcement("No objects detected.")
        return
    parts = []
    label_map = getattr(self.node, "label_map", {})
    profiles = getattr(self.node, "profiles", {})
    for det in detections.detections:
        if det.results:
            best = max(det.results, key=lambda r: r.hypothesis.score)
            raw_label = best.hypothesis.class_id

```

```

        canonical = label_map.get(raw_label, raw_label)
        profile = profiles.get(canonical)
        label = (profile.display_name or canonical) if profile else canonical
        score = round(best.hypothesis.score, 2)
        parts.append(f"{label} {score}")
    text = "I see: " + ", ".join(parts) if parts else "No objects detected."
    self.publish_announcement(text)

```

The fallback chain is: `display_name` → canonical label → raw class ID. If `dome_vision` never loaded (profiles dict is empty), the raw class ID is used as-is.

flowchart LR

```

    A[raw class_id] --> B{label_map?}
    B -- yes --> C[canonical]
    B -- no --> C
    C --> D{profile?}
    D -- yes --> E[display_name or canonical]
    D -- no --> E
    E --> F[announce]

```

Observations

- `getattr(self.node, "latest_detections", None)` and `getattr(self.node, "label_map", {})` are defensive — both work whether oak and `dome_vision` are running or not.
- `count_objects` is registered but unimplemented. Should be removed or implemented.
- The lazy from `std_srvs.srv import Trigger` import avoids pulling `relpy` at module load, keeping the class testable without ROS.
- Calling unconditionally when `service_is_ready()` is `False` is a deliberate bet against transient startup delays. If the service is genuinely absent the future silently stalls — no user-visible error. A timeout mechanism would improve this.

IntentPublisher

`dome_control/commands/intent_publisher.py` is the thin layer between `CommandDispatcher` and the `/intent` ROS2 topic. It accepts an injected `publish_fn` so unit tests stay No-ROS, while production uses `IntentApi` lazily.

Interface

```

class IntentPublisher:
    def __init__(self, publish_fn: Callable[[str], None] | None = None): ...

```

```
def publish(self, name: str, source: str = "cli", slots: dict | None = None) → str | None: ...
def get_api(self): ...
```

Publish Path

```
def publish(self, name: str, source: str = "cli", slots: dict | None = None) → str | None:
    payload = json.dumps({"name": name, "source": source, "slots": slots or {}})
    if self.publish_fn is not None:
        self.publish_fn(payload)
        return None
    return self.get_api().publish(name, source, slots or {})
```

Two paths:

- **Test path** (`publish_fn` injected): JSON payload is passed to the injected function (e.g. `list.append`). Returns `None` — tests assert on the captured JSON, not the return value.
- **Production path** (`publish_fn=None`): delegates to `IntentApi.publish`, which may wait for a /announcement reply for query intents and return the reply text. `get_api()` creates `IntentApi` lazily — avoids importing `rclpy` at module load.

Testing Pattern

```
published = []
pub = IntentPublisher(publish_fn=published.append)
dispatcher = CommandDispatcher(rc, intent_publisher=pub)
dispatcher.dispatch_text("scene describe")
assert json.loads(published[0])["name"] == "describe_scene"
```

The list collects raw JSON strings. Tests parse them to assert on structure.

CommandDispatcher Integration

`CommandDispatcher.__init__` accepts `intent_publisher=None`. When `None`, `publish_intent` creates an `IntentPublisher()` inline (no injection, production default). When injected, the passed instance is used for all intent publishes.

Observations

- `get_api()` creates a new `IntentApi(ConfigManager())`. In a long-running process, the `api` attribute caches the instance after first use.
- The split between `IntentPublisher` (JSON serialization + injection seam) and `IntentApi` (actual ROS2 publisher) exists so `IntentPublisher` remains importable without ROS2.

SpinSurvey: 360° Spin State Machine

SpinSurvey drives a robot in a 360° in-place spin so a perception pipeline can build a full-room view. It has no ROS2 imports — it is pure Python business logic. The caller (a ROS2 node) pumps `tick()` at a fixed rate and publishes the returned velocity.

Two Spin Modes

Continuous (`step_angle_rad=0`): publishes `angular_velocity` every tick until `elapsed_s` exceeds `total_angle_rad / angular_velocity`. Fastest full rotation; blurry frames are possible.

Step-spin (`step_angle_rad>0`): alternates between a spin phase and a pause phase.

spin phase: rotate `step_angle_rad` at `angular_velocity`
pause phase: hold `0.0` for `pause_s` seconds
repeat until `total_angle_rad` accumulated

Default `step_angle_rad=1.047` (60°) with `pause_s=1.0` gives the camera a clean stationary window each step.

Multi-Pass

`pass_count>1` runs multiple full rotations. Between passes, an optional inter-pass offset rotation (`pass_offset_rad`) repositions the starting angle to reduce overlap with the previous pass. If `pass_offset_rad` is 0 the offset defaults to `step_angle_rad / 2`.

State Variables

```
self.elapsed_s          # time accumulated in current pass
self.angle_rotated      # angle in current step (resets after each step)
self.is_pausing         # True during pause phase
self.pause_elapsed_s    # time elapsed in current pause
self.pass_num           # which pass (1-based)
self.is_inter_pass      # True during inter-pass offset rotation
self.inter_pass_elapsed_s
```

tick() Return Contract

```
vel, done = survey.tick(elapsed_s)
# vel: angular_velocity_rad_s (spinning) or 0.0 (pausing / done)
# done: True when all passes complete
```

Caller should stop its timer when `done` is `True`. SpinSurvey itself does nothing with the timer.

Lifecycle

```
SpinSurvey( ... )
├─ ▼ survey.start()           ← reset all state, return initial vel
├─ ▼ survey.tick(dt) × N     ← pump at fixed rate
└─ (vel=0, done=True)       ← all passes complete
```

`start()` is idempotent — calling it again resets the survey and returns the first velocity.

Observations

- `pass_done()` checks accumulated `elapsed_s` against `total_angle_rad / angular_velocity_rad_s`. Because ticks are discrete, the survey may overshoot slightly before `is_done()` returns `True`. The effect is one tick period (200ms at `TICK_HZ=5`).
- Inter-pass offset duration is computed from `pass_offset_rad / angular_velocity_rad_s` — so a larger velocity makes the inter-pass shorter in time but the same in angle.

SpinSurveyNode: Survey Trigger and Tick Driver

`SpinSurveyNode` is a regular ROS2 Node (not a lifecycle node) that wraps `SpinSurvey`. It is always included in the full-stack launch and stays idle until the `/survey/start` service is called.

Design Decision: Service-Triggered

The node does nothing at startup. It waits for a `Trigger` service call on `/survey/start`. This matches the real use case: the robot launches, localizes, then a CLI or behavior triggers the survey explicitly. A lifecycle-managed auto-start would require external activation logic for no benefit.

Published Topics

Topic	Type	When
<code>/cmd_vel</code>	<code>geometry_msgs/Twist</code>	Every tick while running (angular.z only)
<code>/spin_survey/paused</code>	<code>std_msgs/Bool</code>	Every tick (True during pause phases)

Topic	Type	When
/spin_survey/done	std_msgs/Bool	Once, on survey completion

Tick Rate

TICK_HZ = 5 — 200ms per tick. At angular_velocity=0.3 rad/s and step_angle_rad=0.872 rad (50°), each step takes ~2.9s = ~14 ticks. The tick rate is low enough to not stress the executor and high enough for smooth pause detection.

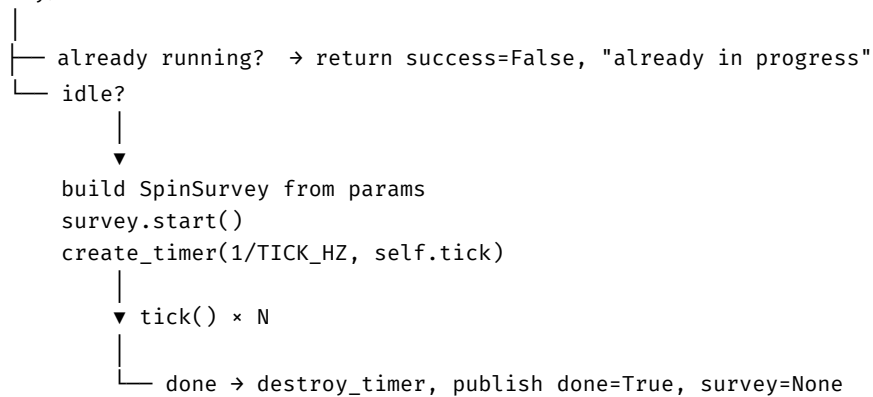
Parameter Defaults

All parameters come from SpinSurveyConfig:

Parameter	Default	Meaning
angular_velocity	0.3	rad/s spin speed
total_angle	6.2832 (2)	full 360° pass
step_angle_rad	1.047	60° per step
pause_s	1.0	seconds per pause
pass_count	1	number of passes
pass_offset_rad	0.0	inter-pass offset (0 → step/2)

Survey Flow

/survey/start called



Integration with dome_vision

SemanticMapNode subscribes to /spin_survey/done and uses it as the trigger to persist the semantic map. dome_vision/launch/full_stack.launch.py and robot.launch.py both include SpinSurveyNode from the dome_control package.

Parameter Definition (Appendix)

`parameter_def.py` defines a single dataclass describing one parameter a command can accept.

```
@dataclass
class ParameterDef:
    name: str          # kwarg name passed to RobotController method
    param_type: type    # e.g. float, bool, str – used for coercion
    required: bool      # if True, dispatcher raises ValueError when absent
    default: object     # None means "no default" (not the same as required=False)
    description: str    # shown in help text
```

`param_type` is the type itself (e.g. `float`), not a string, so `CommandDispatcher` can call `param_type(value)` directly. `bool` needs a special branch because `bool("false") == True`.

No business logic. No imports beyond `stdlib`. Everything else depends on this.

Command Definition (Appendix)

`command_def.py` defines a single dataclass describing one entry in the command registry.

```
@dataclass
class CommandDef:
    method_name: str          # method to call on RobotController
    parameters: list[ParameterDef] # ordered – position maps to CLI arg position
    description: str          # shown in help text
    group: str                # e.g. "movement", "launch", "config"
```

`method_name` decouples the CLI command name from the Python method name. The ordering of parameters is load-bearing: `SimpleCLI._map_to_dispatcher_format` maps CLI positional args to parameter names by index.

No business logic. No imports beyond `parameter_def`.

Control Commands (Appendix)

`control_commands.py` defines two basic robot control commands.

```
"robot.stop": CommandDef("stop_robot", [], "Stop robot movement", group="control")
"robot.status": CommandDef("get_robot_status", [], "Get current robot status", group="control")
```

Neither command takes parameters. `robot.stop` publishes a zero-velocity `Twist` via `MovementApi`. `robot.status` returns a structured status dict including speed config, tracked launch process PIDs, and which API nodes are initialised.

Navigation Commands (Appendix)

`navigation_commands.py` defines three map management commands.

```
"map.save":      CommandDef("map_save",      [], "Save current map to maps/ folder", group="map")
"map.list":      CommandDef("list_maps",      [], "List available maps in maps/ folder", group="map")
"map.serialize": CommandDef("map_serialize", [], "Save current map in SLAM Toolbox serialized form")
```

All three are parameterless — they read `map_name` from the config variable rather than from a CLI argument. `map.save` calls `nav2_map_server map_saver_cli`; `map.serialize` calls the SLAM Toolbox `serialize` service. Both are implemented in `RobotController` using `ProcessApi.run_command_sync`.

Intent Commands (Removed in F15/T07)

`intent_commands.py` defined CLI commands that published to `/intent` via `RobotController` methods (`publish_intent_stop`, `publish_intent_explore`, etc.). These were entries in the `CommandDispatcher` registry.

The file was deleted in F15/T07. Intent routing now lives in `BEHAVIOR_COMMANDS` in `command_dispatcher.py`, handled by `dispatch_text` without going through the registry. `IntentPublisher` replaced the `RobotController` intent methods.

See `12-command_dispatcher.md` for the current routing mechanism.

Semantic Commands (Removed in F15/T07)

`semantic_commands.py` defined user-friendly aliases (`scene.describe`, `scene.count`) that mapped to the same `RobotController` methods as the `intent.*` commands. These were entries in the `CommandDispatcher` registry.

The file was deleted in F15/T07. The `scene.*` aliases survive as entries in `BEHAVIOR_COMMANDS` in `command_dispatcher.py`, routed by `dispatch_text`.

See `12-command_dispatcher.md` for the current routing mechanism.

Audio Feedback (Appendix)

`audio_feedback.py` provides a single lightweight beep function for voice state transitions. It synthesizes a short sine wave and plays it through the configured ALSA output device.

Design: aplay over PyAudio

The initial implementation used `PyAudio` to play tones. `PyAudio`'s device indexing does not map 1:1 to ALSA card numbers, and its initialization floods

stderr with ALSA/JACK enumeration messages. More importantly, playing to the wrong device index silently produces no sound.

The replacement uses `aplay` — the same path used by Piper TTS for speech output:

```
def beep(frequency: int = 880, duration: float = 0.15, device_index: int = 0) → None:
    alsa_device = os.environ.get("SPEECH_ALSA_DEVICE", "")
    sample_rate = 16000
    n = int(sample_rate * duration)
    samples = bytearray(n * 2)
    amplitude = 0.2
    for i in range(n):
        s = int(math.sin(2 * math.pi * frequency * i / sample_rate) * 32767 * amplitude)
        struct.pack_into("<h", samples, i * 2, max(-32768, min(32767, s)))

    cmd = ["aplay", "-q", "-f", "S16_LE", "-r", str(sample_rate), "-c", "1"]
    if alsa_device:
        cmd.extend(["-D", alsa_device])
    try:
        subprocess.run(cmd, input=bytes(samples), check=False, capture_output=True)
    except Exception as exc:
        import sys
        print(f"beep: aplay failed: {exc}", file=sys.stderr)
```

`SPEECH_ALSA_DEVICE` is the same environment variable consumed by `speech_output_node`, so both speech and beeps route to the same output device without separate configuration.

Signal Generation

The sine wave is generated sample-by-sample using `struct.pack_into` directly into a `bytearray`. This avoids a numpy dependency. Amplitude is 20% of full scale — audible but not startling.

Usage in VoiceInputNode

Event	Frequency	Duration
Wake word detected	880 Hz	20 ms
Intent recognized	330 Hz	20 ms
Empty / unrecognized turn	220 Hz	150 ms

The `device_index` parameter is retained for API compatibility but unused; ALSA device selection is done via the environment variable.

Observations

- Beep is synchronous — it blocks the voice loop for `duration` seconds. For the 20 ms beeps this is negligible; the 150 ms failure beep adds latency.
- `check=False` and the broad `except Exception: pass` ensure a broken audio device never crashes the voice pipeline.

Describe Scene Stub Node (Appendix)

`describe_scene_stub_node.py` is a temporary placeholder for the real `/describe_scene` service.

```
class DescribeSceneStubNode(Node):
    def __init__(self):
        super().__init__("describe_scene_stub")
        self.service = self.create_service(Trigger, "/describe_scene", self.describe_scene)

    def describe_scene(self, request, response):
        response.success = True
        response.message = "I see 2 cups and 1 can"
        return response
```

Returns a hardcoded string. Purpose: allows the behavior manager pipeline (`BehaviorManagerNode` $\rightarrow$ `/describe_scene` $\rightarrow$ `/announcement` $\rightarrow$ `SpeechOutputNode`) to be smoke-tested without the actual object detection stack running.

This node should be replaced with a real `/describe_scene` service backed by `/targets/confirmed` from the `oak_roboflow` detection pipeline. See `current.md` next steps.

SimpleCommandParser (Removed in F15/T05)

`simple_parser.py` was the text-to-command parser for the CLI. It converted free-form strings into `ParsedCommand` objects. The file was deleted in F15/T05.

What Replaced It

`CommandDispatcher.dispatch_text(text)` absorbed all functionality:

- Abbreviation resolution (`ABBREV_TO_FULL`, `FULL_NAMES`, `resolve_keyword`)
- Subcommand detection (checks `FULL_NAMES`, abbreviation expansion, and command registry)
- Value type inference (`parse_value: bool  $\rightarrow$  int  $\rightarrow$  float  $\rightarrow$  str`)

`SimpleCLI.execute_command` now calls `dispatcher.dispatch_text(input_text)` directly. The intermediate `_map_to_dispatcher_format` step was also removed.

See `12-command_dispatcher.md` for the current implementation.

Module Entry Point: `__main__.py`

Purpose

`__main__.py` makes the `dome_control` package directly runnable via `python3 -m dome_control`. It owns the top-level lifecycle: initialize `rclpy`, run the CLI, and shut down cleanly regardless of how the CLI exits.

Design

The file is intentionally minimal. All CLI logic lives in `dome_control.interface.simple_cli`; all ROS2 node logic lives deeper in the package. This module's only responsibility is to bracket the CLI call with `rclpy.init()` / `rclpy.shutdown()`.

```
#!/usr/bin/env python3
import rclpy
import dome_control.interface.simple_cli as cli

def main():
    rclpy.init()
    try:
        cli.main()
    finally:
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

The `try/finally` guarantees `rclpy.shutdown()` runs even if `cli.main()` raises — preventing resource leaks in the underlying ROS2 middleware.

Observations

- No issues. File is appropriately thin.
- `rclpy.init()` here means no test can import this module without a live ROS2 environment. Tests should import `simple_cli` directly, not `__main__`, which is already the pattern used in this repo.

SurveyApi: `/survey/start` Service Client

`SurveyApi` is a thin ROS2 Node that calls the `/survey/start` Trigger service on behalf of the CLI. It follows the same pattern as other `*Api` classes: construct, call, return (success, message).

Usage

```
api = SurveyApi()
ok, msg = api.start(timeout_s=5.0)
```

Implementation

```
class SurveyApi(Node):
    def __init__(self):
        super().__init__("survey_api_client")
        self._client = self.create_client(Trigger, "/survey/start")

    def start(self, timeout_s=5.0) → tuple[bool, str]:
        if not self._client.wait_for_service(timeout_sec=timeout_s):
            return False, "/survey/start service not available"
        future = self._client.call_async(Trigger.Request())
        rclpy.spin_until_future_complete(self, future, timeout_sec=timeout_s)
        if not future.done():
            return False, "Service call timed out"
        result = future.result()
        return result.success, result.message
```

`wait_for_service` blocks up to `timeout_s` for `SpinSurveyNode` to appear. `spin_until_future_complete` blocks for the round-trip. Both timeouts use the same value. The caller (`RobotController.start_survey`) wraps the result in a `CommandResponse`.

Notes

- `SurveyApi` is created lazily by `RobotController` (same pattern as `MovementApi`, `SpeechApi`).
- If `SpinSurveyNode` is not running, `wait_for_service` returns `False` after timeout and an error message is returned to the CLI.

SurveyCommands: `survey.start` CLI Command

`survey_commands.py` registers the `survey.start` command in the CLI dispatcher.

Command Definition

```
"survey.start": cd.CommandDef(
    method_name="start_survey",
    parameters=[],
    description="Start a 360° spin survey to build the semantic map",
    group="survey",
)
```

No parameters — the survey params come from the node's ROS2 parameters at launch time.

Dispatch Chain

```
user types: "survey start"
|
▼ CommandDispatcher.dispatch_text
  → resolve_keyword("survey") → "survey"
  → resolve_keyword("start") → "start"
  → candidate "survey.start" found in registry
|
▼ CommandDispatcher.execute("survey.start", {})
|
▼ RobotController.start_survey()
|
▼ SurveyApi.start()
|
▼ /survey/start Trigger service → SpinSurveyNode
```

"sta" is an alias for "start" in ABBREV_TO_FULL, so survey sta also works.